

Chapter3 Application Layer

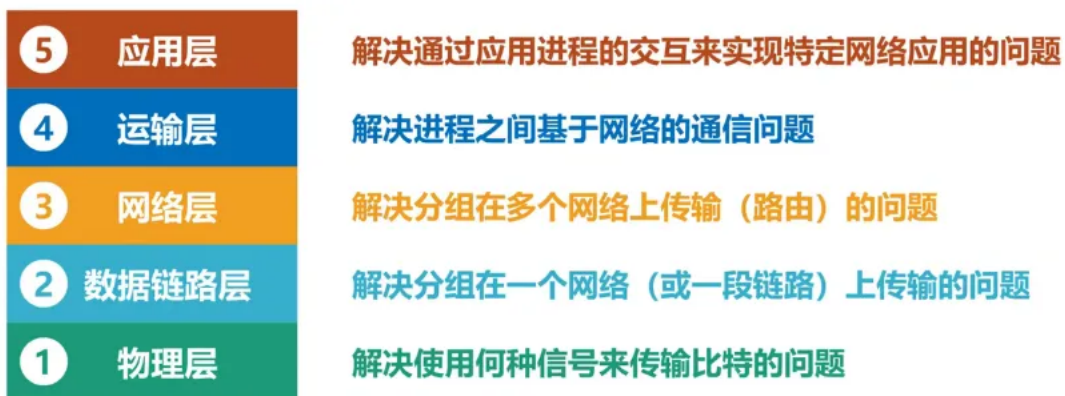
一、Principles of network applications

Outline

- ☐ Conceptual and implementation aspects of application-layer protocols 概念/实现
- ☐ Examining famous protocols.

b站网课

应用层概述



应用层是计算机网络体系结构的最顶层，是设计和建立计算机网络的最终目的。

- 万维网WWW
- 多媒体网络应用
- 文件传送FTP和P2P文件共享
- 电子邮件
- 域名系统DNS
- 动态主机配置DHCP

客户/服务器方式（C/S）和对等方式（P2P）

网络应用程序运行在处于网络边缘的不同端系统上，通过通讯共同完成某种任务。

开发一种新的网络应用首先要考虑的问题—网络应用程序在各个端系统上的组织方式和关系。

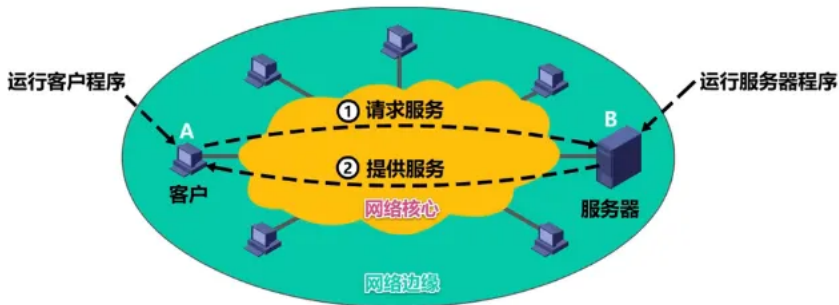
1. 客户/服务器（Client/Server, C/S）

客户和服务指通信中所涉及的两个应用进程。

进程服务和被服务的关系

客户是服务的请求方，服务器是服务的提供方。

服务器总是处于运行状态，并等待客户的服务请求。服务器具有固定端口号（HTTP-80），而运行服务器的主机也有固定的IP地址。



eg. 万维网WWW、电子邮件、文件传输FTP...

服务及中型：服务集中在服务型计算机（<客户计算机）

一台服务器计算机要为多个客户及提供服务→服务器计算机跟不上众多客户机请求。

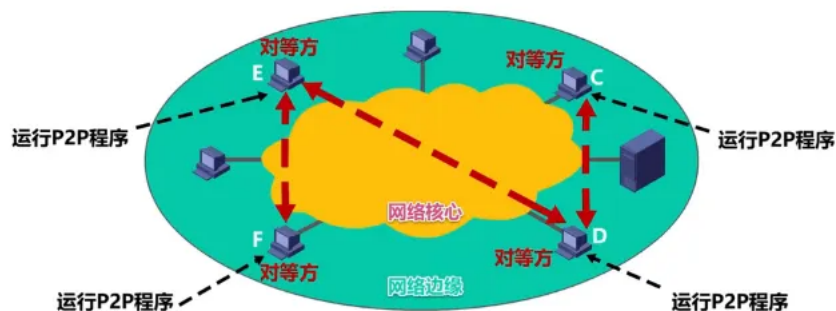
Solution → 用计算机集群构建一个强大的虚拟服务器。

2. 对等 (Peer to Peer, P2P)

没有固定的服务请求者/服务者。

分布在网络边缘各端系统中的应用进程是对等的，被称为对等方。

对等方相互之间通信。



eg. P2P文件共享、即时通信、P2P流媒体、分布式存储.

服务分散型：分散在大量对等计算机中，这些计算机并不为服务提供商所有，而是为个人控制的桌面计算机和笔记本电脑，通常位于住宅、校园和办公室中。

可扩展性：

- P2P方式的最突出特性之一就是它的**可扩展性**。因为系统每增加一个对等方，不仅增加的是服务的请求者，同时也增加了服务的提供者，**系统性能不会因规模的增大而降低**。

成本优势：

- P2P方式**具有成本上的优势**，因为它通常不需要庞大的服务器设施和服务器带宽。为了降低成本，服务提供商对于将P2P方式用于应用的兴趣越来越大。

1. Creating a network APP

编写程序的要求：

- 在（不同的）终端系统上运行 – Run on (different) end systems.
- 通过网络进行通信

不需要为网络核心设备编写软件：

No need to write software for **network-core devices**.

- 网络核心设备不运行用户应用程序

网络核心设备指的是诸如路由器、交换机等基础网络设备，它们的主要功能是转发数据包，维持网络的连通性和效率。

用户应用程序，也称为客户端应用程序，是指运行在终端设备上，直接为用户提供功能和服务的软件。

- 终端系统上的应用程序允许快速开发和传播应用程序
 - 终端系统指的是用户直接使用的计算设备，如个人电脑、智能手机、平板电脑等。这些设备运行各种用户应用程序，例如网页浏览器、电子邮件客户端、移动应用等。
 - 在终端系统上开发应用程序的一个重要优势是：开发者可以专注于直接为用户提供功能和服务，而不必考虑网络传输的细节问题。这种方式允许应用程序的快速开发和传播，因为终端系统上有丰富的开发环境和工具，且更新和部署更为便捷。

Network APP：网络应用程序

1. run on different hosts (end system)
2. communication over network

2. Application architectures 应用架构

- 客户端-服务器（Client-Server）：

客户端设备（如用户的电脑或手机）通过网络与服务器进行通信。服务器提供资源和服务，而客户端向服务器请求这些资源和服务。

- **对等网络（Peer-to-Peer, P2P）：**

各个节点（如用户的电脑）既可以作为客户端，也可以作为服务器。每个节点可以相互提供资源和服务，不需要集中式的服务器。

- **混合架构（Hybrid of Client-Server and P2P）：**

这种架构结合了客户端-服务器和对等网络的特点，利用两者的优势。例如，一些资源和服务由集中式服务器提供，而其他资源和服务由对等节点提供。

2.1. Client-server architecture 客户端-服务器架构

Server :

- ❖ Always-on 在线 host
- ❖ **Permanent IP address** 固定的IP地址，便于客户端查找
- ❖ Data centers for scaling
(可扩展的数据中心) - 服务器通常位于数据中心，可以根据需求进行扩展。

Clients:

- ❖ Contact, Communicate with server
- ❖ May be intermittently 间歇 connected
- ❖ May have dynamic 动态 IP addresses
- ❖ Do not communicate directly with each other 客户端设备之间通常不直接通信，而是通过服务器进行数据交换。

优缺点 (Pros and cons) :

- **基础设施密集型 (Infrastructure intensive)：** 构建和维护客户端-服务器架构需要大量的基础设施，如服务器和数据中心。
- **成本高昂 (Costly to provide)：** 提供和维护这种架构的成本较高。
- **易于管理和安全 (Easy to manage and secure)：** 由于集中管理，客户端-服务器架构相对容易管理和确保安全性。

- ▼ “可扩展的数据中心”指的是**数据中心**能够灵活地增加或减少计算资源（如服务器、存储设备、网络设备等）

数据中心（Data Centers）

- **定义**：数据中心是一个**集中的设施**，用于存放和管理计算资源、存储设备、网络设备和其他相关基础设施。—— 物理的场所。
- **作用**：为服务器提供稳定的运行环境，包括电力供应、网络连接、温度控制和安全保障。

扩展性（Scalability）

- **水平扩展（Horizontal Scaling）**：通过增加更多的服务器来处理更多的请求。每个新增加的服务器可以分担一部分负载，从而提升整个系统的处理能力。
 - **举例**：在高峰期，电商网站可以增加更多的服务器来处理大量的访问请求，避免服务器过载。
- **垂直扩展（Vertical Scaling）**：通过增加现有服务器的硬件资源（如CPU、内存、存储）来提升其处理能力。
 - **举例**：为数据库服务器增加更多的内存和处理器，以便更快速地处理查询请求。

2.2. P2P architecture

- **无始终在线的服务器（no always-on server）**：对等网络中没有中央服务器，所有节点都是平等的。
- **任意终端系统直接通信（arbitrary end systems directly communicate）**：各个节点可以直接与其他节点通信，而不需要通过中央服务器中转。
- **对等节点请求和提供服务（peers request service from other peers, provide service in return to other peers）**：节点可以同时作为服务请求者和服务提供者，相互之间进行资源共享。
- **自我扩展性（self scalability）**：每增加一个新节点，不仅带来新的服务需求，也增加了服务能力，网络可以自动扩展。
- **节点间歇性连接和IP地址变化（peers are intermittently connected and change IP addresses）**：节点可能会间歇性连接到网络，并且可能会频繁更改IP地址。

间歇性连接：在对等网络中，节点（如个人电脑、手机等）不需要始终保持在线状态。

IP地址动态变化？

- ▼ 对等网络（P2P）中节点的**动态变化**，如**频繁更换IP地址和间歇性连接**，会使得节点的**查找和连接变得更加复杂**，相比于具有固定IP地址和持续在线的服务器，确实增加了难度。BUT!!! 这种动态变化也带来了独特的好处（了解）

动态变化的好处

1. 高扩展性（Scalability）

- **资源增长与节点数量成正比**：每当新的节点加入网络时，它不仅消耗资源，也**提供资源**（如带宽、存储、计算能力）。这意味着，网络的整体资源随着节点的增加而线性增长。
- **无中心瓶颈**：没有单一的中央服务器，避免了服务器成为性能瓶颈或单点故障的风险。

2. 资源利用率高

- **充分利用闲置资源**：大量个人设备的计算能力和存储空间往往未被充分利用。P2P网络能够整合这些闲置资源，提高整体资源利用率。
- **降低成本**：无需昂贵的服务器和数据中心，参与者只需贡献自己的设备资源，降低了运营和维护成本。

3. 抗故障性强（Fault Tolerance）

- **去中心化设计**：没有中心服务器的依赖，使得网络对单点故障免疫。即使部分节点离线，网络仍能继续运行。
- **动态自愈**：网络能自动调整和重组，以适应节点的加入和离开，保持服务的连续性。

4. 提高传输效率

- **就近连接**：节点可以与地理位置更接近的其他节点通信，减少数据传输的延迟和网络拥塞。
- **分布式负载**：数据传输的负载分散在众多节点之间，避免了集中式服务器过载的问题。

5. 增强隐私和匿名性

- **分散性**：由于数据分布在众多节点上，追踪和监控特定用户的活动变得更加困难，增加了隐私保护。

应对动态变化的挑战

确实，节点的动态变化增加了查找和连接的复杂性。但P2P网络采用了一系列技术来应对这些挑战：

1. 分布式哈希表（DHT）

- **高效查找机制**：DHT使得在一个巨大且动态变化的节点集合中，仍然可以在 $O(\log N)$ 的

时间复杂度内找到所需的资源或节点。

- **自动更新**：当节点加入或离开时，DHT会自动更新路由信息，确保查找路径的有效性。

2. 节点发现和路由协议

- **先进的算法**：协议如Kademlia、Chord、Pastry等，设计了高效的节点查找和路由算法，能够快速适应网络的动态变化。
- **路由冗余**：维护多个备选路径，即使一些节点失效，也能通过其他路径找到目标节点。

3. 数据冗余和副本

- **提高可用性**：将数据存储在多个节点上，即使部分节点离线，数据仍然可以从其他节点获取。
- **自动副本迁移**：当节点离开网络时，数据副本会自动迁移到其他在线节点。

4. 心跳机制和节点状态监控

- **定期检测**：节点之间定期发送心跳消息，检测彼此的在线状态，及时更新网络拓扑。
- **快速反应**：一旦检测到节点离线，立即调整路由和数据存储策略，最小化影响。

与固定服务器的比较

- **固定服务器的优势**：易于管理和维护，具有稳定的IP地址，方便客户端访问。
- **P2P网络的优势**：扩展性强、成本低、资源利用率高、抗故障能力强。

在一些应用场景下，如大型文件共享、内容分发、分布式计算等，P2P网络的优势非常明显。例如：

- **文件共享应用**：如BitTorrent，当用户数量增加时，下载速度反而会提升，因为每个用户都贡献了上传带宽。
- **分布式计算**：利用大量节点的计算能力，完成复杂的计算任务，如SETI@home等项目。

优点和缺点 (Pros and cons)

- **优点：**

高度可扩展性 (Highly scalable)：对等网络可以通过增加节点来轻松扩展，不受中央服务器的限制。

- **缺点：**

管理难度大 (Difficult to manage)：由于没有中央服务器，管理和维护网络更加复杂。

安全挑战 (Challenge to secure such as privacy risk, online attacks, etc.)：对等网络面临隐私风险、网络攻击等安全问题。

▼ 抗单点故障/管理复杂性–矛盾吗？

去中心化设计的优势与挑战

优势：

- **抗单点故障**：由于没有中央服务器，对等网络中每个节点都是平等的。如果某些节点下线，其他节点可以继续运行，整个网络的功能不会受到严重影响。这种设计使网络具有很高的容错性和可靠性。

挑战：

- **管理复杂性**：正因为没有中央服务器，缺乏一个集中管理的控制点，网络的管理和维护变得更加复杂。协调众多动态变化的节点，需要更加高级的协议和机制。

为什么这并不矛盾？

这其实是同一枚硬币的两面。去中心化设计带来了抗故障的优势，但也引入了管理上的难度。这并不矛盾，而是设计选择的结果。

- **抗故障性与管理性之间的权衡**：去中心化提高了网络的健壮性，但缺乏中央控制意味着需要更复杂的分布式算法来协调节点。

- **复杂性源于自由度**：更多的自由度（节点可以自由加入、离开、变化IP等）使网络更灵活，但同时也增加了保持网络一致性和稳定性的难度。

举个例子

想象一下，一个自发组织的社区：

- **没有领袖（中央服务器）**：每个人都有平等的发言权，社区不会因为某个人的缺席而瘫痪。这就是去中心化的好处。
- **组织协调**：由于缺乏集中领导，社区需要更多的沟通和协调来达成共识，活动的组织也更具挑战性。这对应了管理的复杂性。

3. Processes communicating

▼ Process: Program running instantiated 实例化 as a process

1. **程序**：这是静态的代码，通常存储在硬盘上，包含了指令和数据。
2. **实例化**：当你启动一个程序时，操作系统会为它分配资源（如内存、CPU时间等），并创建一个进程。
3. **进程**：这是程序在运行时的实例。它包含了程序的代码、当前活动的指令、使用的资源（如内存、文件句柄等）以及执行状态（如寄存器值、程序计数器等）

正在执行的program→Process

Process communication:

- Within same host -- inter-process communication (ipc)
- Different hosts -- communicate by exchanging **messages**.

Client process: process that initiates communication,

the peer that is downloading the file is labeled as the client

在文件共享应用中，下载文件的对等方被标记为客户端。

Server process: process that waits to be contacted

the peer that is uploading the file–server

在文件共享应用中，上传文件的对等方被标记为服务器端。

4. Socket

参考黄老师笔记

二、Web and HTTP

1. review

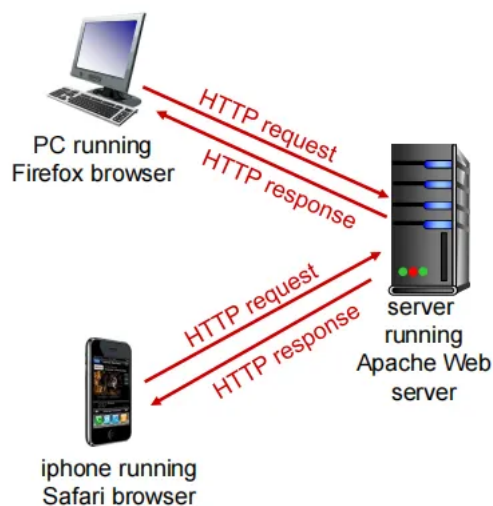
- Web page consists of objects
 - An object is simply a file, can be HTML file, JPEG ,image, Java applet, audio file,...

Web page consists of a **base HTML-file** which includes **several referenced objects** , each object is addressable by a **URL**,

In the context of the web, anything that can be uniquely identified and accessed using a *Uniform Resource Locator (URL)*

2. HTTP overview

HTTP: hypertext transfer protocol – application layer



2.1. HTTP如何使用TCP（传输控制协议） – reliable

- 客户端启动 initiates 一个TCP连接（创建一个套接字 creates socket）到服务器的80端口。
- 服务器接受来自客户端的TCP连接。

- HTTP消息（应用层协议消息）在浏览器（HTTP客户端）和网络服务器（HTTP服务器）之间交换。
- TCP连接关闭。

结合Non-persistent与persistent HTTP

2.2. HTTP is "stateless" 无状态的

Server maintains no information about past client requests

3. HTTP connections

3.1. non-persistent HTTP

非持久性HTTP

- **连接特性**：每个TCP连接最多只能发送一个对象。
- **连接关闭**：对象传输完成后，连接会关闭。
- **多对象下载**：下载多个对象需要多个连接。multiple connection

3.2. persistent HTTP (default)

持久性HTTP

multiple objects can be sent over single TCP connection

参考黄老师笔记

三、Electronic mail

1. Brief introduction

One of the *oldest* and the *most important* applications in the Internet.

因特网上最早流行的一种应用，并且仍然是当今因特网上最重要、实用的应用之一

传统的电话通信属于**实时通信**，而电子邮件与邮政系统的寄信相似

- ① 发件人将邮件发送到自己使用的**邮件服务器**；
- ② 发件人的邮件服务器将收到的邮件按其目的地址转发到收件人邮件服务器中的收件人邮箱；
- ③ 收件人在方便的时候访问收件人邮件服务器中自己的邮箱，获取收到的电子邮件。

asynchronous–异步的 communicate medium

fast, inexpensive, and easy to distribute

电子邮件采用客户/服务器方式

2. Three major components:

- a. user agents 用户代理
 - b. mail servers 邮件服务器
 - c. simple mail transfer 电子邮件所需协议
protocol: SMTP (Simple Mail Transfer Protocol)
-

2.1. User agent = mail reader

用户代理是 用户与电子邮件系统 的接口，又称为电子邮件客户端软件

- Composing 写作、排版 , editing, reading mail messages
- eg. Outlook
- Outgoing, incoming messages stored on server 存储在服务器上的传出和传入消息

2.2. Mail server

邮件服务器是电子邮件系统的基础设施。因特网上所有的ISP都有邮件服务器，其功能是发送和接收邮件，同时还要负责维护用户的邮箱。

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

邮件服务器：

- 邮箱 (Mailbox) ：包含用户的接收邮件。
- 邮件队列 (Message queue) ：存储待发送的外发邮件。

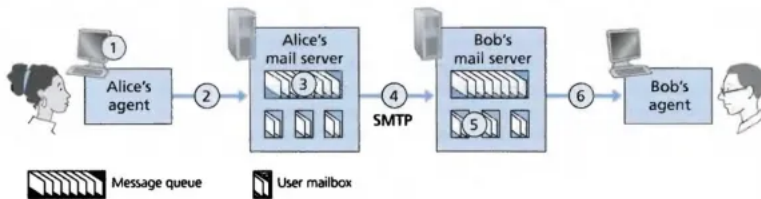
2.3. protocol

包括发送协议 (SMTP) 和读取协议 (POP3,IMAP)

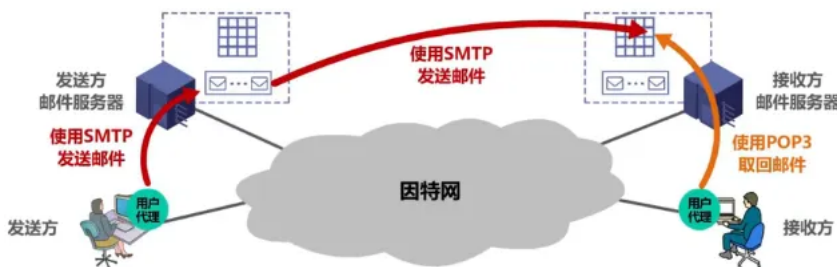
A protocol used to send and receive email messages between mail servers.

▼ 图中Alice给Bob发邮件：

1. 用户代理 (Alice's agent)：用户Alice的邮件客户端，开始创建和发送邮件。
2. Alice的邮件服务器：发送邮件的邮件服务器，将邮件加入发送队列。
3. SMTP协议：邮件通过SMTP协议传输，在Alice的邮件服务器与Bob的邮件服务器之间进行交换。
4. Bob的邮件服务器：接收邮件的邮件服务器将邮件存储在Bob的邮箱中。
5. Bob的邮件客户端 (Bob's agent)：Bob的邮件客户端最终从服务器中读取邮件。



- **client**: sending mail server
- **server** : receiving mail server
- **SMTP** uses **TCP** as its underlying transport protocol to provide the reliable data transfer service. SMTP使用TCP作为其底层传输协议来提供可靠的数据传输服务。



2.3.1. SMTP (Simple Mail Transfer Protocol)

- 建立TCP连接：Client's SMTP mail server establishes a **TCP connection** to the recipients SMTP server **using Port 25**
- 直接传输–Direct transfer: sending server to receiving server

The email is transferred directly from the **sending server** to the **receiving server** without intermediaries.

- **Three phases** of transfer 传输的三个阶段
 - handshaking (greeting) –初始连接阶段，客户端和服务端进行身份识别并建立通信
 - transfer of messages 消息传输–发送设计的邮件消息的阶段
 - closure关闭–消息传输完成后，连接终止
- Command/response interaction (like HTTP, FTP) **命令/响应交互**

- **commands**: ASCII text 由客户端发送的 ASCII 文本，用于执行诸如发送邮件的操作
- **response**: status code and phrase 由服务器发送的状态代码和短语，用于指示命令的结果。

- Messages must be in 7-bit ASCII

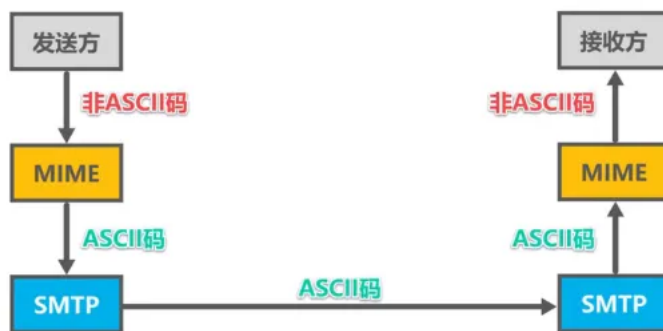
不能传送可执行文件或者其他的二进制对象

SMTP不能满足传送多媒体邮件（例如带有图片、音频或视频数据）的需要。并且许多其他非英语国家的文字（如中文、俄文、甚至带有重音符号的法文或德文）也无法通过SMTP传送

为解决不能传送非ASCII码文本的问题，提出MIME

- ▼ MIME（多用途互联网邮件扩展）

MIME (Multipurpose Internet Mail Extensions)



- ☐ 增加了5个新的邮件首部字段，这些字段提供了有关邮件主体的信息。
- ☐ 定义了许多邮件内容的格式，对多媒体电子邮件的表示方法进行了标准化。
- ☐ 定义了传送编码，可对任何内容格式进行转换，而不会被邮件系统改变。
- 实际上，MIME不仅仅用于SMTP，也用于后来的同样面向ASCII字符的HTTP。

- SMTP uses **persistent connections** 持久连接
 - If the sending mail server has several messages to send to the same receiving mail server, it can send all of the messages over the same TCP connection.

如果发送邮件的邮件服务器有多个邮件要发送到同一个接收邮件服务器，它可以通过同一个TCP连接发送所有邮件。

- SMTP requires message (header & body) to be in 7-bit ASCII

SMTP要求邮件（包括头部和正文）使用7位ASCII编码。

这保证了邮件在不同的系统和平台之间能够兼容并且传输一致。

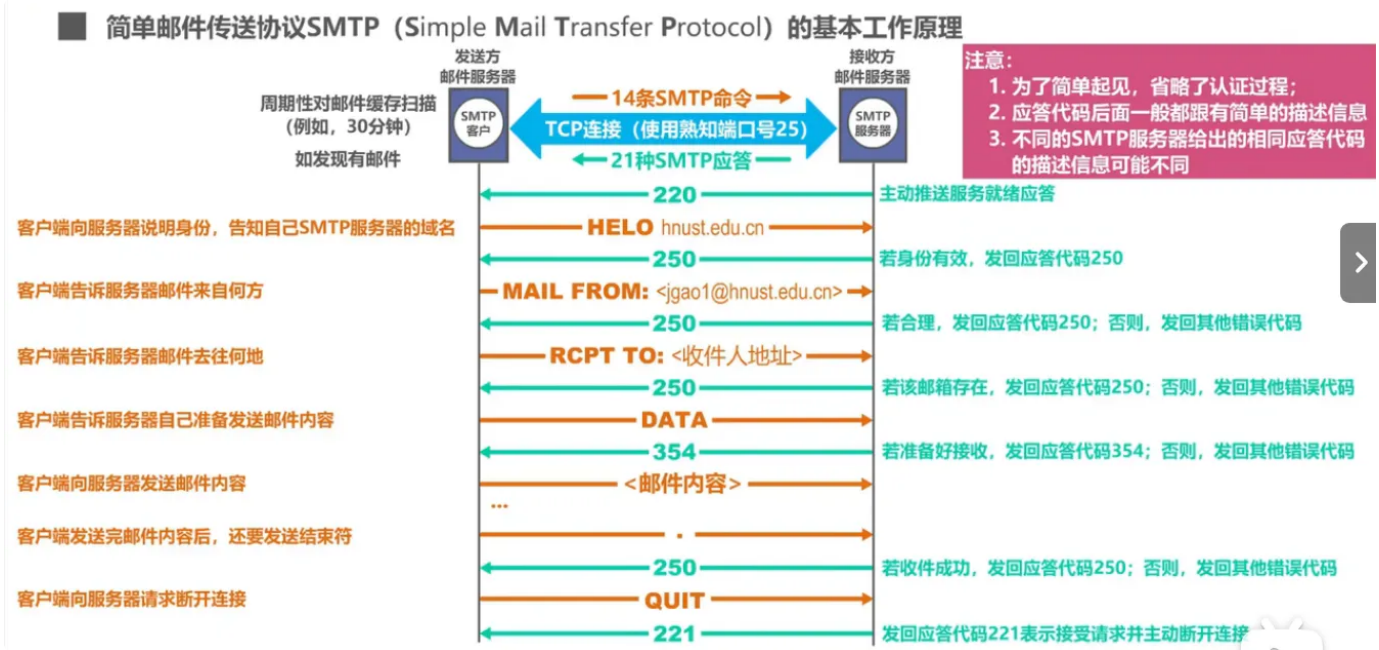
- SMTP server uses **CRLF.CRLF** to determine end of message (where CR and LF stand for carriage return and line feed, respectively)

SMTP服务器使用CRLF.CRLF用于确定消息的结束，
其中CR代表回车（Carriage Return）\r，LF代表换行符（Line Feed）\n。
CRLF表示回车（CR）和换行符（LF）的组合，这在计算机网络协议中常用于标记文本的结束或换行。在SMTP协议中，每一行的消息都会用CRLF标记结束，以确保数据传输的规范性。

SMTP interaction / SMTP交互示例

发送方邮件服务器周期性扫描邮件缓存，如果发现有待转发的邮件 → 建立TCP连接

建立连接后：



以下对话从TCP连接建立开始：

```
1  S: 220 hamburger.edu
2  C: HELO crepes.fr
3  S: 250 Hello crepes.fr, pleased to meet you
4  C: MAIL FROM: <alice@crepes.fr>
5  S: 250 alice@crepes.fr... Sender ok
6  C: RCPT TO: <bob@hamburger.edu>
7  S: 250 bob@hamburger.edu... Recipient ok
8  C: DATA
9  S: 354 Enter mail, end with "." on a line by itself
10 C: Do you like ketchup?
11 C: How about pickles?
12 C: .
13 S: 250 Message accepted for delivery
14 C: QUIT
15 S: 221 hamburger.edu closing connection
```

1. S: 220 hamburger.edu

- 服务器 (S) 向客户端 (C) 发送欢迎信息, 表示SMTP服务已准备好开始通信。

2. C: HELO crepes.fr

- 客户端 (Alice的邮件服务器) 发送HELO命令, 向服务器问候, 并表明自己的身份 (`crepes.fr`) 。

3. S: 250 Hello crepes.fr, pleased to meet you

- 服务器回应确认, 表示已识别并接受来自 `crepes.fr` 的连接请求。

4. C: MAIL FROM: `alice@crepes.fr`

- 客户端发送邮件的发件人地址, 告诉服务器邮件来源是 `alice@crepes.fr` 。

5. S: 250 `alice@crepes.fr`... Sender ok

- 服务器确认发件人地址有效。

6. C: RCPT TO: `bob@hamburger.edu`

- 客户端指定收件人地址为 `bob@hamburger.edu` 。

7. S: 250 `bob@hamburger.edu`... Recipient ok

- 服务器确认收件人地址有效。

8. C: DATA

- 客户端发送DATA命令, 告知服务器准备传输邮件内容。

```
1 354 Start mail input; end with <CRLF>.<CRLF>
```

▼ 详解

1. **354 响应代码**：这表示服务器已准备好接收邮件数据。客户端可以开始发送邮件的实际内容，包括邮件头和正文。
2. **Start mail input**：这是服务器告诉客户端可以开始输入邮件数据。
3. **end with <CRLF>.<CRLF>**：这是指示邮件数据结束的方式。
 - **<CRLF>**：表示回车换行（Carriage Return and Line Feed），即 `\r\n`。
 - **.**：表示邮件数据的结束。当客户端发送完邮件内容后，需要在单独的一行上发送一个点（`.`）。
 - **<CRLF>.<CRLF>**：完整的结束标志是回车换行加一个点再加回车换行（`\r\n.\r\n`），表示邮件数据的结束。

9. S: 354

- 服务器回应，若准备好接受，服务器发送应答代码

10. C: Do you like ketchup?

C: How about pickles?

- 客户端输入邮件的内容（例如，询问对方是否喜欢番茄酱和腌黄瓜）。

11. C: .

- 客户端用一个句点“.”表示邮件内容的结束。

12. S: 250 Message accepted for delivery

- 服务器确认邮件已接受并准备发送。

13. C: QUIT

- 客户端发送QUIT命令，表示通信结束。

14. S: 221 hamburger.edu closing connection

- 服务器回应并关闭连接，表明会话结束。

Comparison of HTTP and SMTP

1. Common characteristics:

- Both are **client-and-server** Model
- Both use **the reliable data transfer service of TCP**

- Use **persistent connection**

2. Difference:

- **HTTP: pull protocol** 拉取协议 –someone loads information on a Web server and users use HTTP to pull the information from the server. –意味着用户加载网页时，浏览器使用HTTP协议从服务器拉取数据。例如，用户请求一个网页，服务器返回页面数据。client需要请求
- **SMTP: push protocol** 推送协议 –the sending mail server pushes the file to the receiving mail server. –SMTP是一个推送协议，发送邮件的邮件服务器将邮件“推送”到接收方邮件服务器。即，发送方主动将邮件发送给接收方，而不是由接收方请求。

p.s. client–发送方--client直接发送

- **SMTP** has 7-bit ASCII restriction. **HTTP** does not have this kind of restriction.

SMTP协议要求邮件内容必须使用7位ASCII字符集，这是一种标准化的字符编码方式。而HTTP协议没有类似的限制，可以支持更广泛的字符集（如UTF-8）。

- **HTTP:** each object encapsulated in its own response message

HTTP：每个对象都封装在自己的响应消息中

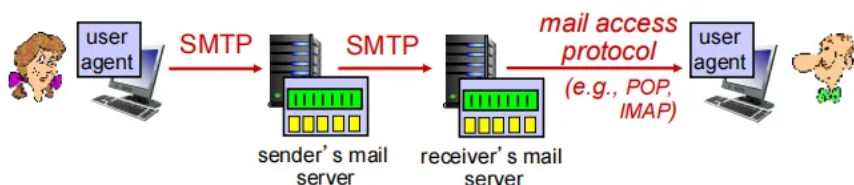
- 在HTTP协议中，服务器每次返回的响应消息通常只包含一个对象，例如一个HTML页面、一个图片文件等。

- **SMTP:** place all of message's objects into one message

SMTP：将所有邮件内容封装到一个消息中

- SMTP协议会将所有的邮件对象（如邮件头、正文等）整合到一个邮件消息中一起传输。

2.3.2. Mail access protocols



SMTP: delivery/storage to receiver's server

Mail access protocol: retrieval/检索 from server

POP: Post Office Protocol 邮局协议[RFC 1939]: authorization, download

从邮件服务器检索邮件，主要实现授权和下载功能

- ▼ 特点：下载后，邮件通常会从服务器上删除，适合离线邮件访问

POP（邮局协议）设计的初衷是为了在本地存储邮件，从而实现离线访问。以下是详细解释：

POP协议与离线邮件访问

1. **本地存储**：当使用POP协议时，邮件客户端会从邮件服务器上下载邮件并将其存储在本地设备上（如计算机、手机等）。
2. **删除服务器邮件**：下载完成后，通常会将服务器上的邮件删除。这确保了邮件只存在于本地设备上。
3. **离线访问**：由于邮件已经下载到本地设备，无需与服务器保持连接，用户可以在没有互联网连接的情况下访问、阅读和管理邮件。

优点

- **带宽节省**：下载一次后，无需重复从服务器获取邮件，减少了带宽消耗。
- **快速访问**：由于邮件存储在本地，访问速度更快。
- **离线操作**：用户可以在没有互联网的情况下查看和处理邮件，这对于网络不稳定或无网络环境下非常有用。

注意事项

- **同步问题**：由于POP协议默认删除服务器上的邮件，在多个设备上使用时可能会导致邮件不同步。
- **数据丢失风险**：如果本地设备丢失或损坏，可能会丢失所有邮件。因此，定期备份是必要的。

通常使用端口**110**（POP3）

❑ **邮局协议POP**（Post Office Protocol），POP3是其第三个版本，是因特网正式标准。

非常简单、功能有限的邮件读取协议。用户只能以**下载并删除方式**或**下载并保留方式**从邮件服务器下载邮件到用户方计算机。**不允许用户在邮件服务器上管理自己的邮件。**（例如创建文件夹，对邮件进行分类管理等）。

POP3 protocol

POP3协议主要用于电子邮件的接收和管理，它的工作包括：

- 在认证阶段验证用户身份。
- 在事务阶段，客户端可以列出邮件、检索邮件、删除邮件等。
- 该协议支持简单的命令与响应机制，确保邮件的有效管理。

POP3 protocol

Port: 110

authorization phase

- ❖ client commands:
 - **user**: declare username
 - **pass**: password
- ❖ server responses
 - **+OK**
 - **-ERR**

transaction phase, client:

- ❖ **list**: list message numbers
- ❖ **retr**: retrieve message by number
- ❖ **dele**: delete
- ❖ **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

端口: 110

认证阶段 (Authorization phase)

- 客户端命令:
 - **user**: 声明用户名
 - **pass**: 提供密码
- 服务器响应:
 - **+OK**: 成功
 - **-ERR**: 错误

事务阶段 (Transaction phase)

- 客户端命令:
 - list**: 列出邮件编号
 - retr**: 按编号检索邮件
 - dele**: 删除邮件
 - quit**: 退出

▼ 解释

1. 认证阶段：

- 客户端首先通过**user**命令提供用户名，然后通过**pass**命令提供密码，服务器响应**+OK表示认证成功，或者-ERR**表示认证失败。

2. 事务阶段：

- 客户端在认证成功后，可以使用多种命令与服务器交互：
 - **list**：列出所有邮件的编号。
 - **retr**：检索指定编号的邮件内容。
 - **dele**：删除指定编号的邮件。
 - **quit**：结束会话，退出POP3协议。

3. 响应：

- 服务器会相应地返回邮件编号列表、邮件内容或状态信息，并在完成操作后返回**+OK表示成功，或者-ERR**表示出错。

- 4. 在事务阶段中，客户端可以对邮件进行操作（如查看和删除），这些操作会在会话结束时提交。

先前的示例使用了POP3的“下载并删除”模式

- Bob如果更换客户端，无法重新读取邮件。

1. POP3的“下载并保留”模式

在此模式下，邮件的副本会保留在服务器上，可以在不同的客户端之间访问。

2. POP3在会话之间是无状态的

- 每个POP3会话的操作都是独立的，不会记住之前的会话状态。

IMAP: Internet Mail Access Protocol [RFC 17 30]: more features, including manipulation of stored msgs on server

允许在服务器上管理和组织邮件，支持多设备同步，适合在线邮件访问

143(IMAP)/993(IMAPS,IMAP over SSL/TLS)

- ❑ **因特网邮件访问协议IMAP** (Internet Message Access Protocol) ，IMAP4是其第四个版本，目前还是因特网建议标准。

功能比POP3强大的邮件读取协议。用户在自己的计算机上就可以操控邮件服务器中的邮箱，就像在本地操控一样，因此IMAP是一个联机协议。

POP3和IMAP4都采用基于TCP连接的客户/服务器方法

HTTP: Web-based Email, such as gmail, Hotmail, Yahoo! Mail, etc.

用户可以通过浏览器访问邮件，无需专门的邮件客户端

80(HTTP)/443(HTTPS)

■ 基于万维网的电子邮件

- ☐ 通过浏览器登录（提供用户名和口令）邮件服务器万维网网站就可以撰写、收发、阅读和管理电子邮件。这种工作模式与IMAP很类似，不同的是用户计算机无需安装专门的用户代理程序，只需要使用通用的万维网浏览器。
- ☐ 邮件服务器网站通常都提供非常强大和方便的邮件管理功能，用户可以在邮件服务器网站上管理和处理自己的邮件，而不需要将邮件下载到本地进行管理。

特性	HTTP	IMAP
使用方式	通过Web浏览器访问	通过邮件客户端应用程序访问
示例	Gmail, Hotmail, Yahoo! Mail	Outlook, Thunderbird, Apple Mail
特性	无需安装客户端，所有操作在浏览器中进行	支持多设备同步，允许离线访问和管理邮件
邮件存储位置	云端存储	服务器存储
同步	不支持自动同步，手动刷新获取新邮件	支持多设备间自动同步
离线访问	不支持，需要网络连接	支持，下载后可离线访问
操作在服务器上的同步	不支持，操作本地化	支持，所有操作在服务器上同步更新

▼ 服务器差异

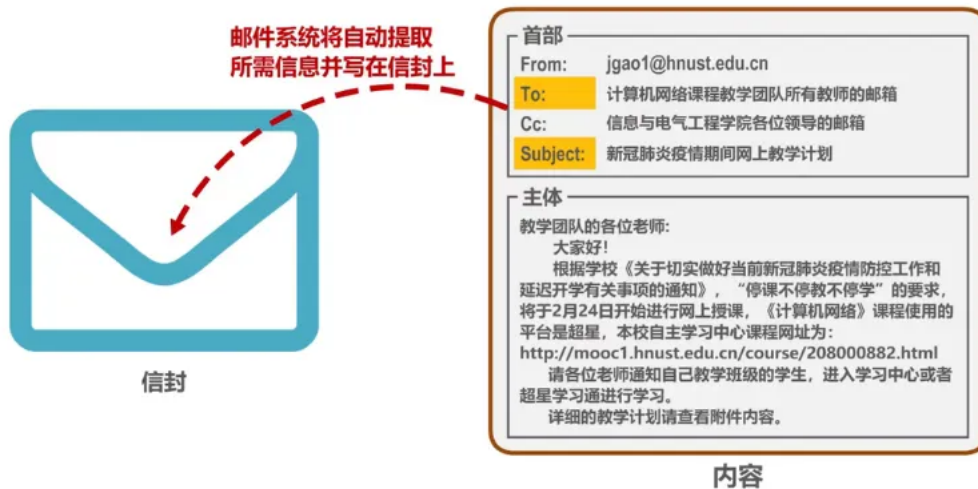
HTTP: 基于Web的电子邮件通常使用Web服务器（例如Apache或Nginx）来提供邮件服务，并通过HTTP/HTTPS协议进行通信。

IMAP: IMAP服务器（例如Dovecot或Courier-IMAP）专门处理电子邮件的存储和检索，并通过IMAP协议进行通信。

3. Mail message format

电子邮件的信息格式并不是由SMTP定义的，而是在RFC 822中单独定义的。这个RFC文档已在2008年更新为RFC 5322。一个电子邮件由信封和内容两部分。而内容又由首部和主题两部分构成

ASCII characters only



SMTP: protocol for exchanging email msgs

RFC 822: standard for text message format

4. Summary

- 电子邮件系统采用**客户/服务器方式**。三个主要组成构件：**用户代理**，**邮件服务器**，以及电子邮件所需的**协议**。
 - ☐ **用户代理**是用户与电子邮件系统的接口，又称为**电子邮件客户端软件**。
 - ☐ **邮件服务器**是电子邮件系统的基础设施。因特网上所有的ISP都有邮件服务器，其功能是**发送和接收邮件**，同时还要负责维护用户的邮箱。
 - ☐ **协议**包括**邮件发送协议**（例如SMTP）和**邮件读取协议**（例如POP3）。
- 常用的**邮件发送协议**是**简单邮件传送协议SMTP**
 - ☐ 基于TCP连接，端口号为25； ☐ 只能传送ASCII码文本 ☐ 用于用户代理向邮件服务器发送邮件以及邮件服务器之间的邮件发送
- 为解决SMTP传送非ASCII码文本的问题，提出了**多用途因特网邮件扩展MIME**。
- 常用的**邮件读取协议**有以下两个：
 - ☐ **邮局协议POP3**：非常简单、功能有限的邮件读取协议。用户只能以**下载并删除方式**或**下载并保留方式**从邮件服务器下载邮件到用户方计算机。**不允许用户在邮件服务器上管理自己的邮件**。
 - ☐ **因特网邮件访问协议IMAP**：功能比POP3强大的邮件读取协议。**用户在自己的计算机上就可以操控邮件服务器中的邮箱**，就像在本地操控一样，因此IMAP是一个**联机协议**。
 - ☐ POP3和IMAP4都采用**基于TCP连接的客户/服务器方式**。POP3使用端口110，IMAP4使用端口143。
- 基于万维网的电子邮件
 - ☐ 通过**浏览器**登录（提供用户名和口令）**邮件服务器万维网网站**就可以撰写、收发、阅读和管理电子邮件。这种工作模式与IMAP很类似，不同的是用户计算机无需安装专门的用户代理程序，只需要使用通用的万维网浏览器。
 - ☐ 这种工作模式在**用户浏览器与邮件服务器网站之间使用HTTP协议**，而**邮件服务器之间使用SMTP协议**。

Three main component of E-mail system : Users agents, mail server, and SMTP

SMTP and the comparison with HTTP.

SMTP mail message format

Mail **access** protocols : PoP, IMAP, HTTP

四、DNS

1. Hostname/IP Address

域名系统 (Domain Name System)

Internet hosts: two ways to identify a host

- **hostname**, 主机名 e.g., www.yahoo.com
 - Easy to be remembered, human beings prefer
 - Provide little information about the location of this host

–提供关于主机位置的信息很少

- Difficult to process by router – 路由器处理困难

▼ 为什么路由器处理困难？

1. 可变性

- **主机名长度不固定**：主机名的长度和格式不固定，处理起来更加复杂。
- **字符处理**：主机名由字母、数字和其他符号组成，而IP地址由固定格式的数字组成。路由器在处理字符数据时需要更多的计算和存储资源。

2. 性能和效率：

- **速度问题**：直接处理IP地址比处理主机名更快，因为IP地址是固定长度、层次结构的数字格式，便于快速查找和路由。
- **资源开销**：处理主机名需要更多的内存和计算资源，这对路由器的性能产生不利影响。

- **IP address** (IPv4: 4 bytes (i.e.,32 bit) such as 121.7.106.83)
 - Fixed length, hierarchical structure–固定长度、层次结构,
 - Easy to process by router, routers prefer
 - Difficult to be remembered

The transformation between hostname and IP address

主机名和IP地址之间的转换的**必要性**：

■ Applications running at a host need to translate a hostname to its IP address (identifier is IP address + port number) 在主机上运行的应用程序需要将主机名转换为对应的IP地址

DNS –实现主机名和ip地址之间的转换

```
C:\Users\lenovo>ping www.baidu.com
```

```
正在 Ping www.wshifen.com [103.235.47.188] 具有 32 字节的数据 :  
来自 103.235.47.188 的回复: 字节=32 时间=28ms TTL=53  
来自 103.235.47.188 的回复: 字节=32 时间=20ms TTL=53  
来自 103.235.47.188 的回复: 字节=32 时间=33ms TTL=53  
来自 103.235.47.188 的回复: 字节=32 时间=35ms TTL=53
```

我们输入ping的是域名，但是实际上ping到的是ip地址— 即使不使用域名，也可以使用ip地址进行寻址。
但是域名与ip地址相比，便于记忆。-- 用域名访问



2. DNS工作流程

- Before a network application (e.g., web browser) sends a message to its receiving host, it should know the IP address of the receiving host. –
 - 在网络应用（例如：网页浏览器）向接收主机发送消息之前，它必须知道接收主机的IP地址。
- The application invokes the client side of DNS, specifying the hostname that needs to be translated. –应用程序调用DNS客户端，置顶需要转换的主机名
- Then, **DNS client** sends a query message to a **DNS server**.
 - 然后，DNS客户端向DNS服务器发送查询请求。
- After a time delay, DNS client receives a DNS reply message including the IP address for this hostname.
 - 在经历一段时间的延迟后，DNS客户端接收到一个DNS响应消息，其中包含该主机名对应的IP地址。
- The IP address is then passed to the invoking application.
 - 获取到IP地址后，它被传递给调用的应用程序。

■ Once the browser receives the IP address from DNS, it can initiate a TCP connection to the HTTP server process (IP address + port 80)

- 一旦浏览器从DNS接收到IP地址，它便可以发起与HTTP服务器的TCP连接（IP地址 + 端口号 80）。

-
- **网络应用与IP地址获取：** 在发送消息给目标主机之前，网络应用需要知道目标主机的IP地址。通常我们使用域名而不是直接输入IP地址。
 - **DNS的工作流程：** 当你在浏览器中输入网站地址（如 `www.example.com` ），浏览器需要通过DNS查询得到该网站的IP地址。DNS首先接受一个查询请求，通过查询DNS服务器返回正确的IP地址。
 - **DNS的角色：** DNS的作用是将易记的主机名转换为计算机能够处理的IP地址。这个过程是浏览器能够通过TCP协议建立连接的前提。
 - **最后步骤：** 获取到IP地址后，浏览器使用此地址建立与目标服务器的TCP连接，然后通过HTTP协议进行数据交换。
-

3. DNS提供的服务

DNS provides

- Translation between **hostname to IP address**
- **Host aliasing:** translation from alias name别名 to canonical name正式名.

主机别名是将容易记住的**别名转换为正式**的主机名。

- Two types of hostnames
 - Canonical name: formal hostname, e.g., relay1.west-cost.enterprise.com
 - Alias name: easy to remember, e.g., enterprise.com
- **两种主机名：**
 - **规范名（Canonical name）：** 这是正式的主机名，例如： `relay1.west-cost.enterprise.com` 。
 - **别名（Alias name）：** 这是易于记住的名称，例如： `enterprise.com` 。

▼ 区分规范名（正式名称）和别名（简化名称）有几个重要的原因：

- **管理和维护**：规范名通常会反映具体的服务器或服务的详细信息，例如部门名称、服务器位置等。如果使用简化的别名，可能会失去这些详细的信息，降低管理和维护的效率。
- **灵活性**：使用别名允许在后台更改规范名，而无需通知用户。例如，如果服务器名称改变了，只需更新别名的映射，而用户仍然可以使用相同的别名访问服务。这种灵活性在大规模网络管理中非常重要。
- **负载均衡**：别名可以指向多个规范名，实现负载均衡。例如，别名 `www` 可以同时指向多个服务器，这样可以分散流量，提高访问速度和可靠性。
- **易用性**：别名通常更简短，更易记。对于用户来说，输入和记住别名比记住复杂的规范名要容易得多。
- **安全性**：通过使用别名，可以隐藏规范名的具体信息，增加安全性，减少潜在的攻击面

-
- **Mail server aliasing**: translation from alias name to canonical name of a mail server
 - e.g., mail server is mailhost1.ipm.edu.mo, and E-mail address is xuyang@ipm.edu.mo

- **邮件服务器别名：**

邮件服务器的别名是将别名转换为邮件服务器的正式名称。

- 例如：邮件服务器的正式地址是 `mailhost1.ipm.edu.mo`，而电子邮件地址是 `xuyang@ipm.edu.mo`。

4. Load distribution 负载分配

- *Replicated web servers* for **busy sites**: a set of IP addresses for one canonical name – 繁忙网站的复制网站服务器：一个规范名称对应多个IP地址
 - Busy sites are replicated 自我复制 over multiple web servers. 繁忙的网站通过多个网站服务器进行复制。
 - Each replicated server run on a different end system and have a different IP address. – 每个复制的服务器在不同的终端系统上运行，具有不同的IP地址。
 - But all replicated servers are associated with one canonical name. –但所有复制的服务器都与同一个规范名称关联
- **DNS rotation** distributes the traffic among the replicated servers.

DNS轮询（DNS rotation）分配流量到复制的服务器

- When *clients* make a *DNS query* for a busy site, the server responds with the entire set of IP addresses, but rotates the ordering of the addresses within each reply

当客户端对繁忙站点进行DNS查询时，服务器会返回整个IP地址集合，但每次[回复中的地址顺序会进行轮换](#)。

这种方式可以将流量分配到不同的复制服务器上，确保负载均衡。

- **复制网站服务器：**

当一个网站的流量过大时，通常会通过**复制多个服务器来分担流量**。每个服务器会有不同的IP地址，但是它们都与同一个规范的主机名（例如 `www.example.com`）关联。这样做可以减少单个服务器的负载，提高网站的访问速度和可靠性。

- **DNS轮询：**

在多个复制服务器的情况下，当客户端请求访问该网站时，DNS服务器会返回多个IP地址。为了避免单一服务器过载，DNS服务器会根据**轮询**的方式改变返回IP地址的顺序，从而均匀地**分配流量到不同的服务器上**。这样，每次请求都可以分散到不同的服务器，优化了网站的性能并提高了访问速度。

- **负载均衡：**

这种机制有助于避免单个服务器过载，提高网站的可用性和响应速度

- ▼ **负载**

系统或设备所承受的工作量或处理的任务数量。负载可以有多种形式，例如CPU负载、网络负载、服务器负载等。

- **CPU负载：**

CPU负载指处理器正在执行任务的数量。高CPU负载意味着处理器正在处理大量任务，可能会导致性能下降。

- **网络负载：**

网络负载指网络设备（如路由器、交换机）处理的数据流量。高网络负载可能会导致网络拥塞和延迟。

- **服务器负载：**

服务器负载指服务器正在处理的请求数量。高服务器负载可能会导致响应时间变慢或服务器崩溃。

5. 如何实现DNS

Q：因特网是否可以只是用一台DNS服务器？

A：No！因为因特网的规模很大，这样的域名肯定会因为超负荷而无法正常工作，而且一旦域名服务器出现故障，整个因特网就会瘫痪。

How to implement DNS?

Centralized DNS? One DNS server contains all the mappings between the hostnames and IP addresses

- single point of failure
- traffic volume, less scalability
- distant centralized database---a single DNS server cannot be “close to” all end systems, which will leads to long delay for some end systems.
- Maintenance: this centralized database will be huge and has to be updated frequently to account for every new host.

A: *doesn't scale!*

1. 集中式Centralized DNS?

- 一个DNS服务器包含所有主机名hostname与IP地址 IP address的映射。
 - 单点故障：如果这个单一的DNS服务器出现问题，整个系统将无法工作。
 - 流量量大 traffic volume，扩展性差 less scalability：所有请求都集中到一个服务器，会增加负载，无法有效处理大量的查询请求。
 - 远程集中式数据库：单一DNS服务器无法与所有end system终端系统靠得很近，因此对于某些终端系统的查询会有较长的延迟。
 - 维护问题Maintenance：这个集中式数据库将非常庞大，需要频繁更新，以便处理每一个新主机的请求。

结论：这种方式无法扩展（Doesn't scale）。

Distributed DNS: DNS consists of a large number of DNS servers distributed around the globe and specify how the DNS server and querying hosts communicate.

2. 分布式Distributed DNS:

- DNS由全球分布的大量DNS服务器组成，指定了这些DNS服务器和查询主机之间的通信方式。

- **集中式DNS的缺点：** 在集中式DNS架构中，所有DNS查询都由单一服务器处理，这意味着如果服务器出现故障，整个系统将无法使用。而且，由于所有数据都存储在一个服务器上，这导致了流量和维护的瓶颈。此外，集中式服务器通常位于远程位置，可能会造成一些查询的延迟。
- **分布式DNS的优势：** 为了解决这些问题，现代DNS采用了分布式架构，将DNS查询分散到多个全球分布的DNS服务器。这种方式可以减少单一服务器的负担，提升查询效率，同时避免单点故障，提高系统的可靠性和扩展性。

6. DNS具体层次结构

1983，因特网采用**层次结构的命名树**作为主机的名字（即域名），并使用分布式的域名系统DNS。DNS使大多数域名都在本地解析，仅少量解析需要在因特网上通信，因此系统效率很高。

DNS是分布式系统，即使单个计算机出了故障，也不会妨碍整个系统的正常运行。

■ 因特网采用**层次树状结构的域名结构**

■ 域名的结构由若干个分量组成，各分量之间用“点”隔开，分别代表不同级别的域名。

... .三级域名.二级域名.顶级域名

☐ 每一级的域名都由英文字母和数字组成，不超过63个字符，不区分大小写字母。

☐ 级别最低的域名写在最左边，而级别最高的顶级域名写在最右边。

☐ 完整的域名不超过255个字符。

■ 域名系统既不规定一个域名需要包含多少个下级域名，也不规定每一级的域名代表什么意思。

■ 各级域名由其上一级的域名管理机构管理，而最高的顶级域名则由因特网名称与数字地址分配机构ICANN进行管理。

■ **顶级域名TLD (Top Level Domain)** 分为以下三类：

☐ **国家顶级域名nTLD** 采用ISO 3166的规定。如cn表示中国，us表示美国，uk表示英国、等等。

☐ **通用顶级域名gTLD** 最常见的通用顶级域名有七个，即：**com (公司企业)**、net (网络服务机构)、org (非营利性组织)、int (国际组织)、edu (美国教育结构)、gov (美国政府部门)、mil (美国军事部门)。

☐ **反向域arpa** 用于反向域名解析，即IP地址反向解析为域名。

■ 在**国家顶级域名下注册的二级域名均由该国家自行确定**。例如，顶级域名为jp的日本，将其教育和企业机构的二级域名定为ac和co，而不用edu和com。

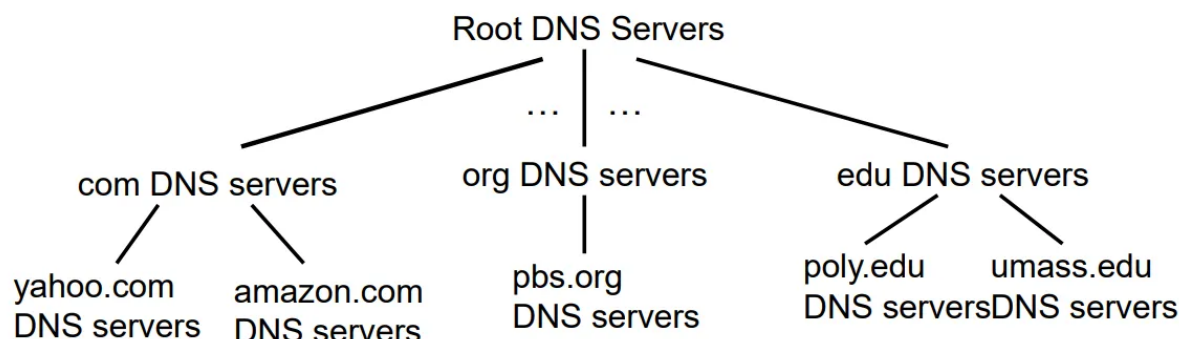
■ 我国则将**二级域名**划分为以下**两类**：

☐ **类别域名** 共七个：ac (科研机构)、**com (工、商、金融等企业)**、edu (教育机构)、gov (政府部门)、net (提供网络服务的机构)、mil (军事机构)和org (非营利性组织)。

☐ **行政区域名** 共34个，适用于我国的各省、自治区、直辖市。例如：bj为北京市、sh为上海市、js为江苏省，等等。

注意区分

这种按等级管理的命名方式，便于维护名字的唯一性，并且也容易设计出一种高效的域名查询机制。需要注意的是，域名是一个逻辑概念，并不代表计算机所在的物理地点。



client wants IP for www.amazon.com; 1st approx:

- ❖ client queries **root server** to find com DNS server
- ❖ client queries .com DNS server (**TLD server**) to get amazon.com DNS server
- ❖ client queries amazon.com DNS server (**authoritative server**) to get IP address for www.amazon.com

1. DNS的层次结构:

- **根 DNS服务器**: 根DNS服务器位于系统的顶部, 负责管理不同**顶级域名** (如.com, .org, .edu等) 的DNS服务器。
- **顶级域 DNS服务器 (TLD DNS)**: 例如, .com域名下有多个DNS服务器, 处理像 yahoo.com、amazon.com等域名的查询。
- **域名服务器**: 每个网站都有特定的DNS服务器, 比如amazon.com的DNS服务器。

2. 客户端查询过程:

- 客户端希望获取**www.amazon.com
- **第一步**: 客户端首先查询 root server **根DNS服务器**, 获取**.com**域名的DNS服务器。
 - **第二步**: 客户端查询**.com DNS服务器**, 这个被称为**顶级域 (TLD) 服务器**, 来获取 amazon.com的DNS服务器。
 - **第三步**: 客户端查询amazon.com DNS服务器, 即**权威DNS服务器**, 最终获取 ****www.amazon.com**的IP地址**。

-
- **分布式结构**: DNS采用了**分布式和层次化**的结构, 这使得域名解析可以更加高效地进行。根DNS服务器是整个系统的顶点, 它负责管理不同的域名层级。每个域名 (如.com) 都有自己的DNS服务器, 这些服务器负责将域名解析为具体的IP地址。
 - **查询过程**: 当客户端想要获取某个域名的IP地址时, 它会通过**层级查询**的方式逐步找到最终的解析结果。首先, 客户端会联系根DNS服务器, 获取顶级域DNS服务器信息, 然后进一步查询特定域名的DNS服务器, 最终得到所需的IP地址。

6.1. Root DNS service: 根域名服务器

- In the Internet, there are **13 root DNS servers** most of which are located in North America. (2012)
- Root DNS servers typically do **not** contain hostname to IP mappings; they **contain mappings for locating top-level domain (TLD) servers**.

根DNS服务器的功能：

- 根DNS服务器通常**不包含**主机名到IP地址的映射（即不直接进行域名解析）。它们主要负责定位顶级域（TLD）服务器的映射。例如，根DNS服务器可以帮助查询“.com”、“.org”以及其他顶级域的DNS服务器。

▼ 根DNS服务器的功能

根DNS服务器是DNS层次结构的顶层。它们不存储特定域名的IP地址信息，而是指向存储这些信息的顶级域（TLD）服务器。

▼ 定位过程

1. 查询启动：

当用户在浏览器中输入一个URL（如 www.example.com）并按下回车键，浏览器会向本地DNS解析器发送查询请求。

2. 本地DNS解析器：

如果本地DNS解析器没有缓存这个域名的IP地址，它会向根DNS服务器发送查询请求。

3. 根DNS服务器响应：

根DNS服务器不直接返回域名的IP地址，而是返回顶级域（如“.com”）DNS服务器的IP地址。

4. 查询TLD服务器：

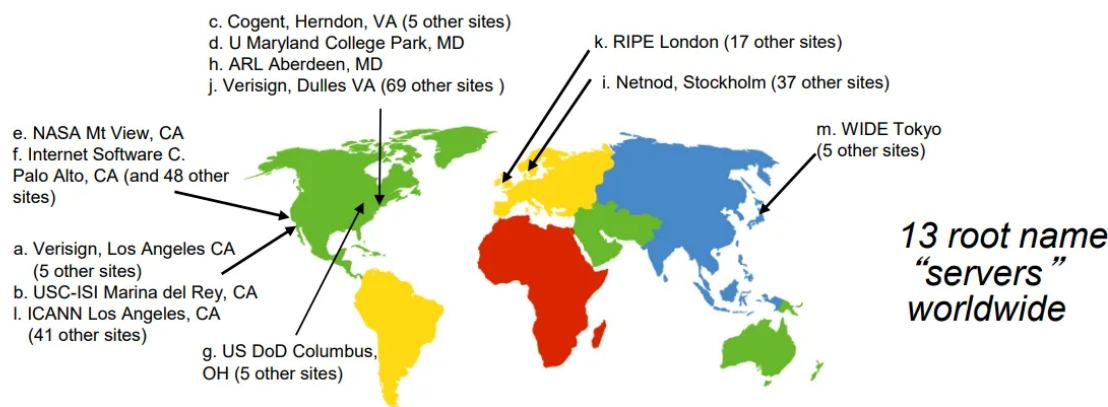
本地DNS解析器接着向顶级域（TLD）服务器发送查询请求。TLD服务器存储有该顶级域中所有二级域名的信息。

5. 获取最终IP地址：

TLD服务器返回负责具体域名（如 www.example.com）的权威DNS服务器的IP地址。

本地DNS解析器最终向权威DNS服务器发送查询请求，获取域名对应的IP地址，并返回给用户的浏览器。

- 1 用户浏览器 → 本地DNS解析器 → 根DNS服务器 → TLD服务器 → 权威DNS服务器 → IP地址返回给用户浏览器



1. 分布位置：

- 这些根DNS服务器分布在全球多个地点，以下是几个例子：
 - Cogent（位于美国弗吉尼亚州Herndon，其他5个地点）
 - U Maryland（位于美国马里兰州College Park）
 - NASA（位于美国加利福尼亚州Mt View，其他48个地点）
 - Verisign（位于美国加利福尼亚州洛杉矶，其他5个地点）
 - RIPE（位于英国伦敦，其他17个地点）
 - Netnod（位于瑞典斯德哥尔摩，其他37个地点）
 - WIDE（位于日本东京，其他5个地点）

2. 根DNS服务器的作用：

- 根DNS服务器的主要作用是[帮助寻找顶级域（TLD）服务器](#)。它们不会直接解析主机名到IP地址的映射，而是提供如何定位顶级域服务器的信息。

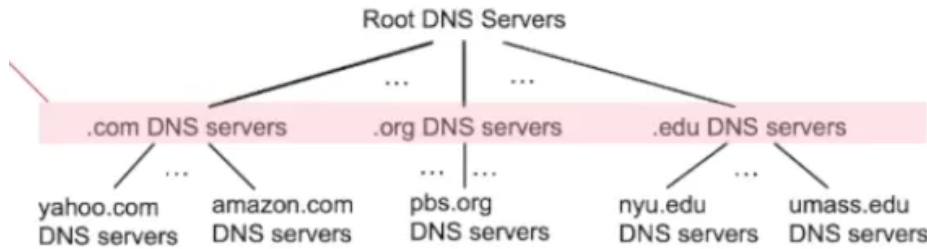
•

6.2. TLD DNS Server：顶级域服务器

- **Top-level domain (TLD) servers** are responsible for com, org, net, edu, etc, and all top-level 通用顶级域名 country domains uk, fr, ca, jp. 各个国家的顶级域
 - Company **Network Solutions** maintains the TLD servers for com top-level domain. – Network Solutions公司维护了 .com 顶级域名的TLD服务器。
 - Company **Educause** maintains the TLD servers for the edu top-level domain. – Educause公司负责维护 .edu 顶级域名的TLD服务器。

- **TLD servers** typically do not contain hostname to IP mappings; they contain

TLD服务器通常不包含主机名到IP地址的直接映射。它们的作用是提供指向**权威服务器 (authoritative servers) **的映射，*权威服务器才会进行具体的域名解析。*



理解：

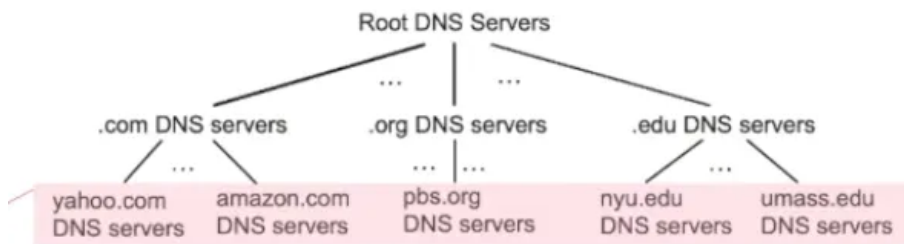
TLD服务器的作用： TLD服务器管理着特定顶级域（如 .com、.org）的名称解析。当一个用户请求访问一个网站时，TLD服务器会指引用户去查询到更具体的域名服务器（如**www.example.com**的权威DNS服务器），*这些服务器负责最终的IP地址解析。*

与根DNS服务器的关系： 根DNS服务器指向TLD服务器，而TLD服务器则指向负责具体网站解析的权威DNS服务器。这样，DNS查询过程通过多个层级逐步实现域名的最终解析。

公司角色： 如 Network Solutions 和 Educause 等公司负责管理和维护相应的TLD服务器，确保顶级域名的管理和查询可以顺利进行。

6.3. Authoritative DNS Servers: 权威DNS服务器

- **Authoritative DNS servers 权威DNS服务器:** provides mapping from **hostname to IP address for organization's servers** (e.g., Web server, mail server). – 通常用于组织的服务器
 - **Store the mapping** from hostname to IP address.
 - Each organization can implement its own authoritative DNS server to hold these records, it also can have these records stored in an authoritative DNS server of some service provider. – 每个组织可以实现自己的权威DNS服务器来存储这些记录，也可以将这些记录存储在某个DNS服务提供商的权威DNS服务器中。
 - Each organization with publicly accessible hosts (e.g., Web servers or mail servers) on the Internet must have records that map the name of these hosts to IP addresses– 任何拥有公开可访问主机（如Web服务器或邮件服务器）的组织，都必须拥有将这些主机名称映射到IP地址的记录。



理解：

权威DNS服务器的作用： 权威DNS服务器负责存储和提供**组织内服务器** 的详细信息，特别是它们的主机名和IP地址之间的映射。例如，一个Web服务器的主机名可能是**[www.example.com](#)**，它的IP地址则会存储在权威DNS服务器中，当用户访问时，DNS服务器会返回相应的IP地址。

组织的DNS管理： 每个组织都可以设置自己的权威DNS服务器来存储这些信息，尤其是在其网络中使用的各种服务。比如，组织可能会有多个Web服务器、邮件服务器等，它们都有对应的主机名和IP地址。

外部DNS服务提供商： 组织还可以选择将这些记录交由外部DNS服务提供商管理和存储，尤其是当组织没有足够的技术资源来维护自己的DNS基础设施时。

公开可访问主机： 如果一个组织的Web服务器或邮件服务器需要向互联网上的其他用户提供服务，它必须确保这些服务器的主机名和IP地址能够通过**权威DNS服务器**进行查询。

简而言之，权威DNS服务器负责在互联网上提供主机名与IP地址之间的最终映射，保证客户端能够找到并访问相应的服务器。

6.4. 本地DNS服务器 (Local DNS Server)

ipconfig

- Each ISP (residential ISP, company, university, etc) has a **local DNS server**. When a host connects to Internet through a ISP, the ISP provides the host with the IP addresses of its local DNS server.
- 每个ISP（互联网服务提供商）（如住宅ISP、公司、大学等）都有一个本地DNS服务器。当一个主机通过ISP连接到互联网时，ISP会提供本地DNS服务器的IP地址。
- A local DNS server **does not** strictly belong to the hierarchy of servers.
- 本地DNS服务器不严格隶属于服务器层级结构。
本地DNS服务器只是用作请求的中介，帮助主机快速解析域名，而不直接位于DNS的层级结构之内。
- A host's local DNS server is typically "close to" the host.
 - For an institutional ISP, at the same LAN
 - For a residential ISP, no more than a few routers.

- 主机的本地DNS服务器通常非常靠近主机。
 - 对于机构级ISP，本地DNS服务器通常位于同一个局域网（LAN）内。
 - 对于住宅级ISP，通常不会跨越多个路由器，只需经过几个路由器。
- A **local DNS server** is also called a **default name server**. When a host makes a DNS query, query is sent to its local DNS server
- 本地DNS服务器，也叫默认名称服务器。当主机进行DNS查询时，查询会被发送到它的本地DNS服务器。
 - The local DNS server acts as a proxy, forwards query into hierarchy of servers.

本地DNS服务器充当代理，将查询转发到层级结构中的其他服务器。

- Reduces lookup latency for commonly searched hostnames

本地DNS服务器能减少常查询主机名的查找延迟。

- The IP address of your local DNS server can be easily determined by using the command **ipconfig /all** in Windows OS
- 如何找到本地DNS服务器的IP地址：
 - 可以通过在Windows操作系统中使用 `ipconfig /all` 命令轻松地获取本地DNS服务器的IP地址。

讲解：

1. **本地DNS服务器的作用：** 本地DNS服务器是计算机或网络中的一个重要组成部分，负责接收并处理从主机发出的DNS查询请求。它通常会作为一个代理服务器存在，并将查询请求转发给更上层的DNS服务器。通过使用本地DNS服务器，常访问的主机名能够被快速解析，减少了延迟。
2. **提高效率：** 本地DNS服务器通常会缓存那些常用的域名解析记录。这样，当同一主机名再次被请求时，服务器不必重新向其他更高层次的DNS服务器查询，而是直接从缓存中提取数据，从而加速了查询响应速度。
3. **如何查找本地DNS服务器：** 用户可以在Windows操作系统中通过 `ipconfig /all` 命令查看网络配置信息，其中包括本地DNS服务器的IP地址。这有助于确认网络的DNS配置以及故障排除。

总之，本地DNS服务器通过缓存和代理功能，加速了DNS查询的过程，尤其对于经常访问的主机名，它大大减少了延迟。

7. A distributed,hierarchical database

▼ 为什么说DNS是数据库？

因为它具有一些与数据库类似的功能和特点。以下是DNS作为数据库的一些关键点：

1. 数据存储

- 记录存储：DNS存储域名与IP地址之间的映射关系。每个域名都对应一个或多个IP地址，存储在DNS记录中。
- 类型多样：DNS记录包括A记录（IPv4地址）、AAAA记录（IPv6地址）、CNAME记录（规范名称）、MX记录（邮件交换服务器）等多种类型。

2. 数据查询

- 查询机制：DNS允许用户和应用程序通过DNS查询来获取存储在其中的数据，例如将域名解析为IP地址。
- 递归查询和迭代查询：DNS支持递归查询和迭代查询，通过逐级查询不同的DNS服务器来获取最终结果。

3. 数据分布

- 分布式数据库：DNS是一个分布式数据库系统，其数据分布在全球各地的多个DNS服务器上。这种分布式架构提高了系统的可靠性和可扩展性。
- 分层结构：DNS具有分层结构，从根DNS服务器到顶级域（TLD）服务器，再到权威DNS服务器，每一层次负责不同范围的数据。

4. 数据管理

- 更新和维护：DNS记录需要定期更新和维护，以确保数据的准确性和及时性。例如，当IP地址发生变化时，需要更新对应的DNS记录。
- 缓存：为了提高查询效率，DNS解析器通常会缓存查询结果，以减少重复查询的负载。

5. 数据一致性

- 数据一致性：尽管DNS是分布式的，但它具有一定的数据一致性机制，确保不同DNS服务器之间的数据同步和一致性。

DNS : A distributed,hierarchical database

The DNS is

- A distributed database 分布式数据库 implemented in hierarchy of DNS servers. (scalability扩展性 and reliability可靠性)
- An application-layer protocol that allows hosts to query the distributed database. 应用层协议：DNS作为一个应用层协议，允许主机查询分布式数据库中的信息。
- An application-layer protocol that uses UDP as its underlying transport layer protocol and use Port 53. DNS使用UDP作为其底层传输层协议，并且使用53端口进行通信。-- Less delay.

Less delay – 但是会有数据错误和丢失 -- multiple DNS query

▼ 更多扩展

由于UDP（用户数据报协议）是无连接协议，不保证数据包的顺序和交付，DNS（域名系统）在使用UDP时引入了一些机制来处理这个问题：

1. **请求标识符（Transaction ID）**：每个DNS请求都有一个唯一的请求标识符，用于区分不同的查询和响应。即使数据包顺序发生变化，也能确保响应与正确的请求对应。
 2. **单一请求-响应**：DNS查询通常是独立的单一请求-响应事务。每个查询从发送到接收到响应都是一个独立的操作，因此不会涉及数据包顺序问题。
 3. **超时和重传**：如果客户端在预定时间内没有收到响应，会重发请求。UDP不会自动重传丢失的数据包，但DNS客户端会负责这一部分工作。
 4. **TCP回退**：在某些情况下，如果UDP不能满足需求（如响应过大或需要确保顺序），DNS会回退到TCP协议。TCP是面向连接的协议，可以保证数据包的顺序和完整性。
- DNS is commonly employed by other application-layer protocols—including HTTP, SMTP, and FTP—to translate user-supplied hostnames to IP addresses.
 - **常用于其他协议**：DNS广泛应用于其他应用层协议中，比如HTTP、SMTP、FTP等，用来将用户提供的主机名（hostname）转换为对应的IP地址。

8. DNS名称解析示例

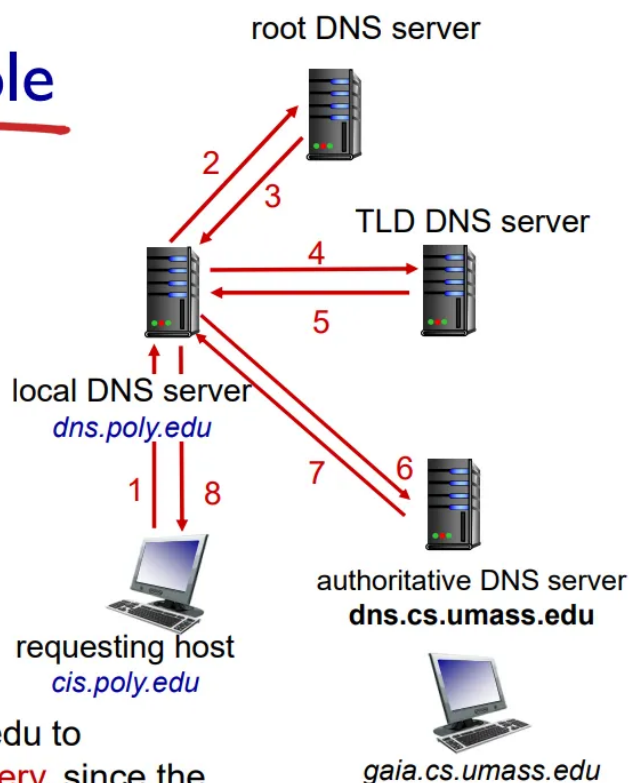
DNS name resolution example

- ❖ host at cis.poly.edu wants IP address for gaia.cs.umass.edu

iterated query:

- ❖ contacted server replies with name of server to contact
- ❖ **8 DNS messages** were sent: 4 query messages and 4 reply messages.

The query sent from cis.poly.edu to dns.poly.edu is a **recursive query**, since the query asks dns.poly.edu to obtain the mapping on its behalf.



Application Layer 2-74

- 主机（cis.poly.edu）需要获取gaia.cs.umass.edu的IP地址。

8.1.1. 迭代查询 (Iterated Query) :

- 被联系的服务器会返回下一个需要联系的服务器的名称。

过程:

- 一共发送了8条DNS消息：4条查询消息和4条回复消息。
 - i. **请求主机（cis.poly.edu）**向本地DNS服务器（dns.poly.edu）发送查询请求。
 - ii. 本地DNS服务器（dns.poly.edu）向根DNS服务器发出查询。
 - iii. 根DNS服务器返回TLD DNS服务器的地址。
 - iv. 本地DNS服务器（dns.poly.edu）再向TLD DNS服务器（负责edu域名的服务器）发送查询请求。
 - v. TLD DNS服务器返回授权DNS服务器的地址。
 - vi. 本地DNS服务器（dns.poly.edu）向授权DNS服务器（dns.cs.umass.edu）发出查询请求。
 - vii. 授权DNS服务器返回目标主机（gaia.cs.umass.edu）的IP地址。
 - viii. 本地DNS服务器将结果返回给请求主机（cis.poly.edu）。

递归查询：

- 从cis.poly.edu到dns.poly.edu的查询是递归查询（**recursive query**）。这是因为该查询请求dns.poly.edu代替它获取最终的映射信息。换句话说，dns.poly.edu会一直向其他服务器查询，直到得到目标主机的IP地址。

理解：

1. 迭代查询：

- 在迭代查询中，每一台服务器只会告诉请求方下一步应该查询哪一台服务器，而不会直接返回最终结果。每个查询都会逐步引导到正确的DNS服务器。

2. 递归查询：

- 与迭代查询不同，递归查询要求DNS服务器代替客户端继续查询，直到返回最终结果。在本示例中，cis.poly.edu请求dns.poly.edu进行递归查询，dns.poly.edu会一直查询，直到最终获得gaia.cs.umass.edu的IP地址。

3. DNS消息的数量：

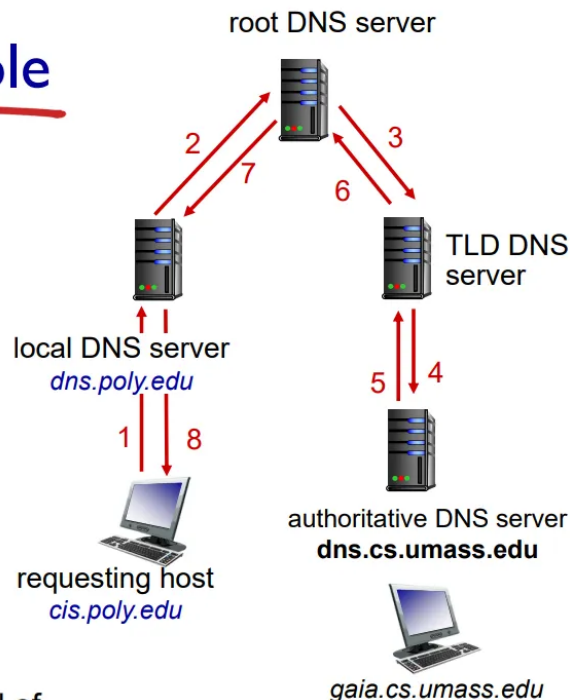
- 本示例中共发送了8条DNS消息，其中4条为查询消息，4条为回复消息。这表明，在查询过程中，每一步都需要一次请求和一次响应。

通过这种方式，DNS可以实现高效的分布式解析，逐层查询直到找到正确的IP地址。

DNS name resolution example

recursive query:

- ❖ puts burden of name resolution on contacted name server
- ❖ heavy load at upper levels of hierarchy?



a DNS query chain for which all of the queries are recursive.

8.1.2. 递归查询 (Recursive Query) :

- 递归查询：把名称解析的责任burden交给被联系的DNS服务器。
- 上层层次的负载较重：递归查询会增加上层DNS服务器的负担，因为它们需要处理多个查询请求。

过程：

- a. 请求主机 (cis.poly.edu) 向本地DNS服务器 (dns.poly.edu) 发送查询请求，要求解析 gaia.cs.umass.edu的IP地址。
- b. 本地DNS服务器向根DNS服务器发送查询请求。
- c. 根DNS服务器返回TLD DNS服务器的地址 (如.edu域的TLD服务器) 。
- d. 本地DNS服务器向TLD DNS服务器发送查询请求。
- e. TLD DNS服务器返回授权DNS服务器的地址。
- f. 本地DNS服务器向授权DNS服务器 (dns.cs.umass.edu) 发送查询请求。
- g. 授权DNS服务器返回gaia.cs.umass.edu的IP地址。
- h. 本地DNS服务器将结果返回给请求主机 (cis.poly.edu) 。

理解：

1. 递归查询的工作原理：

- 递归查询要求DNS服务器从根服务器开始，逐步向下查询，直到找到最终的IP地址，并将结果返回给客户端。
- 在此过程中，客户端将所有查询任务交给本地DNS服务器处理。本地DNS服务器向上层DNS服务器发起请求，一直到获得所需的IP地址为止。

2. 递归查询与迭代查询的不同：

- 迭代查询：每个DNS服务器只会将查询引导到下一个服务器，直到找到最终的答案，而不会替客户端完成所有查询。
- 递归查询：客户端要求DNS服务器替它完成所有查询工作，直到返回最终结果。--本地DNS解析器全权负责整个查询过程

3. 负载问题：

- 递归查询虽然能够减少 客户端的查询次数，但却将负担转移到了上层的DNS服务器。随着查询链条的增加，根DNS和TLD DNS服务器的负担会逐步加重，特别是在互联网规模越来越大时，递归查询会造成上层DNS服务器的高负载。

这种递归查询机制通过分层结构有效地将DNS解析过程分解开来，但在实际应用中可能导致某些服务器的负载过重。

9. DNS：缓存与更新记录

DNS : caching , updating records

- ❖ **Caching**:(any) name server learns mapping, it *caches* mapping
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
 - thus root name servers not often visited
- ❖ cached entries may be *out-of-date* (best effort name-to-address translation!)
 - if name host changes IP address, may not be known Internet-wide until all TTLs expire
- ❖ **Caching** is to improve delay performance by reducing the number of DNS messages.

9.1. 缓存 (Caching) :

- 缓存：任何DNS服务器在学习到主机名与IP地址的映射关系后，都会将其缓存起来。
- Cache entries timeout (disappear) after some time (TTL)
- ▼ TTL(Time To Live)

TTL代表“存活时间”，是DNS记录中的一个重要参数。它指示DNS解析器应该缓存该记录的时间长度，以秒为单位。例如，如果TTL设置为600秒，DNS解析器将在600秒内缓存该记录，然后在缓存过期后重新查询。

用途：

1. **减少查询次数**：通过缓存DNS记录，TTL可以减少DNS服务器的查询次数，减轻服务器的负载。
2. **提高效率**：缓存可以加快DNS解析速度，提高用户访问网站的效率。
3. **控制更新频率**：较短的TTL允许DNS记录更频繁地更新，较长的TTL则可以减少更新频率。

缓存条目的过期 (Timeout) : 缓存内容会在一定时间后过期 (TTL, 生存时间) 。

- TLD servers typically cached in **local name servers**
- ▼ TLD (Top-Level Domain)

TLD代表“顶级域”，是域名系统中的最高层级

TLD服务器缓存：TLD服务器通常将缓存存储在本地名称服务器中，这意味着根名称服务器不会频繁访问。

缓存的条目可能是过时的，因此需要尽力进行地址到名称的翻译。

- 例如，如果主机名改变了IP地址，那么这个改变可能需要直到所有TTL过期后，才能在互联网上广泛更新。

缓存的目的是通过 improve delay performance by reducing the number of DNS messages 减少DNS查询消息的数量来提高延迟性能。通过缓存DNS查询结果，DNS服务器能够更快地响应常见的查询请求，从而减少查询所需的时间。

理解：

1. 缓存机制：

- **缓存映射：**当DNS服务器学习到某个主机名和IP地址的映射后，它会将这个映射存储到缓存中。这样，当以后需要查询该主机名时，服务器可以直接从缓存中获取结果，而无需再次进行复杂的查询过程。

2. 缓存过期与TTL：

- 每个缓存条目都有一个TTL（Time to Live，生存时间），表示该条目存活的最大时间。当TTL过期后，该条目就会从缓存中移除。
- TLD（顶级域名）服务器通常会将这些缓存存储在本地名称服务器中，从而减少对根名称服务器的访问频率。

3. 缓存过时的风险：

- 如果某个主机名的IP地址发生了变化，而该IP地址仍然在缓存中，用户可能会获取到过时的IP地址，导致无法连接到目标服务器。
- 这个问题在缓存过期前会影响多个用户，直到TTL过期后，缓存才会更新。

4. 提高DNS性能：

- 通过缓存机制，可以减少DNS查询所需的时间，并减少DNS服务器之间的通信频率，这对于提高网络的响应速度和减少延迟非常有帮助。

总之，缓存是DNS系统中非常重要的一部分，它能够通过减少DNS查询的频率，优化延迟性能，并提高用户访问网站的速度。然而，缓存的过时也可能带来一些问题，尤其是在主机名IP地址变化的情况下。

9.2. DNS记录 (DNS Records)

DNS: distributed db storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- **name** is hostname
- **value** is IP address

type=NS

- **name** is domain (e.g., foo.com)
- **value** is hostname of authoritative name server for this domain

type=CNAME

- **name** is alias name for some “canonical” (the real) name
- **www.ibm.com** is really **servereast.backup2.ibm.com**
- **value** is canonical name

type=MX

- **value** is name of mailserver associated with **name**

Ttl: time to live.

- **Determines when a RR should be removed from a cache**

DNS: 分布式数据库存储资源记录 (RR)

RR格式: (name, value, type, ttl)

- **name**: 记录的名称
- **value**: 记录的值
- **type**: 记录的类型
- **ttl**: 生存时间 (Time To Live)

记录类型 (Record Types)

1. type=A

- **name** 是主机名 (例如: www.example.com)
- **value** 是与该主机名对应的IP地址 (例如: 192.168.1.1)

2. type=NS

- **name** 是域名 (例如: foo.com)
- **value** 是该域名的权威名称服务器的主机名 (例如: ns1.foo.com)

3. type=CNAME

- **name** 是某个“别名”名称, 指向一个“规范的” (真实的) 名称 (例如: www.ibm.com 实际上指向 servereast.backup2.ibm.com)

- **value** 是规范名称（例如：servereast.backup2.ibm.com）

4. type=MX

- **value** 是与**name**关联的邮件服务器的名称（例如：mailserver.foo.com）
-

TTL (Time to Live)

- TTL：生存时间，指示一个记录在缓存中保存的时间长度。
 - TTL决定了当记录过期时，缓存中何时应被移除。
-

理解：

- **A记录**：将主机名与其对应的IP地址关联起来。当我们访问一个网站时，浏览器会查询DNS中的A记录，以获取该网站的IP地址。
- **NS记录**：指示某个域名（如foo.com）的权威DNS服务器。通常在域名注册时，会配置该记录来指向负责管理该域名的DNS服务器。
- **CNAME记录**：为某个主机名提供别名，常用于指向另一个“规范的”域名。例如，`www.ibm.com` 可以是 `servereast.backup2.ibm.com` 的别名。这种记录常用于负载均衡或通过更简单的域名访问服务。
- **MX记录**：用来指定与某个域名相关的邮件服务器。它通常用于电子邮件系统中，指示邮件应该发送到哪个服务器。
- **TTL**：指示DNS记录在缓存中保存的时长。TTL越短，记录更新的频率越高，但也可能增加查询延迟。TTL过期后，DNS服务器需要重新查询该记录的最新数据。

10. DNS协议与消息格式

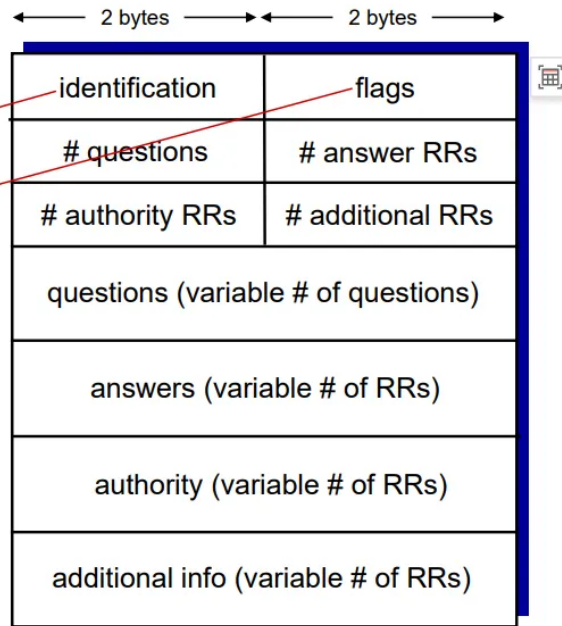
❖ *query* and *reply* messages, both with same *message format*

msg header

- ❖ **identification**: 16 bit # for query, reply to query uses same #

- ❖ **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative

The **first 12 bytes** is the header section



查询（query）和回复（reply）消息：两者具有相同的消息格式

消息头（msg header）

1. 标识符（identification）：
 - 16位标识符，用于查询的标识，查询的回复使用相同的标识符。
2. 标志（flags）：
 - 查询或回复：标志消息是查询还是回复。
 - 递归请求：指示是否需要递归查询。
 - 递归可用：标志服务器是否支持递归查询。
 - 回复是权威的：标志回复是否来自权威DNS服务器。

消息格式的其他部分：

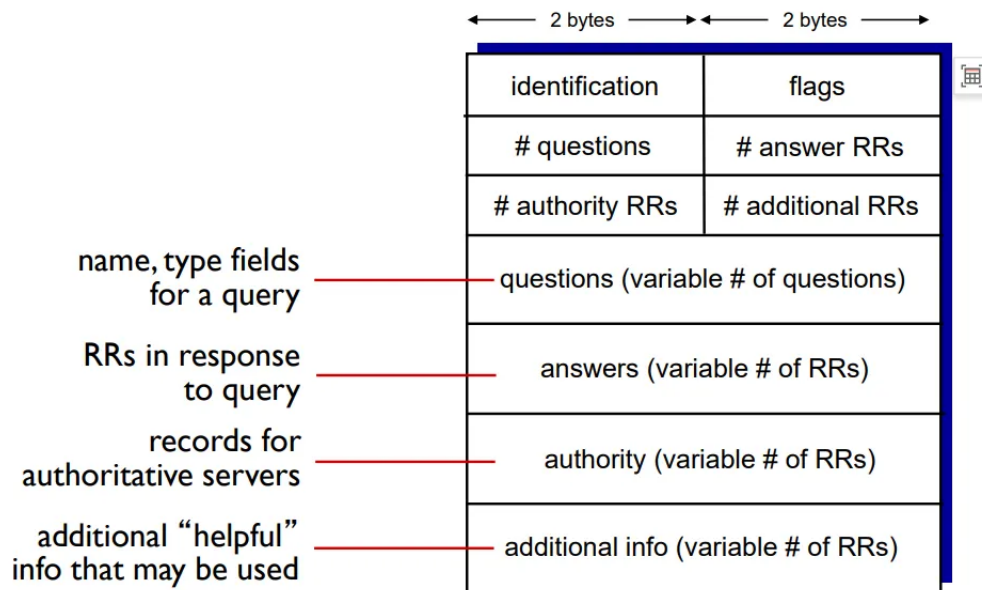
- 问题数（# questions）：查询中问题的数量。
- 回答资源记录数（# answer RRs）：答案中的资源记录数。
- 权威资源记录数（# authority RRs）：包含权威信息的资源记录数。
- 附加资源记录数（# additional RRs）：附加的资源记录数。

第一部分是消息头（12字节）

理解：

- 查询与回复消息：DNS协议中的查询消息与回复消息的格式相同，通过标识符和标志位区分它们。

- **标识符**：查询消息和回复消息通过一个16位的数字来进行关联。每次查询都会生成一个唯一的标识符，服务器回复时会带上相同的标识符，以此来关联查询和回复。
- **标志位（flags）**：
 - **查询或回复**：该标志指示消息是一个查询请求还是一个回复。
 - **递归请求和递归可用**：如果客户端请求递归查询，它会在标志位中设置相应标志，服务器也会表明自己是否支持递归查询。
 - **权威回复**：指示DNS回复是否来自权威DNS服务器。如果是，表示该服务器拥有该域名的管理权。



消息头（msg header）

- **标识符（identification）**：16位标识符，查询使用一个标识符，回复使用相同的标识符。
- **标志（flags）**：包含查询或回复的标志，递归请求和可用性，是否为权威回复等信息。

字段描述：

1. **问题数（# questions）**：该字段指示查询中问题的数量。
2. **回答资源记录数（# answer RRs）**：该字段指示该消息中答案的资源记录数。
3. **权威资源记录数（# authority RRs）**：该字段指示提供权威信息的资源记录数。
4. **附加资源记录数（# additional RRs）**：该字段指示附加的资源记录数。

查询部分（questions）：

- **name, type 字段**：这些字段指定了查询的名称和类型。

响应部分：

- **答复资源记录（RRs）**：提供与查询相关的资源记录。

- 权威服务器的记录 (authority) : 提供有关权威服务器的信息。
 - 附加信息 (additional info) : 该部分提供任何额外的、可能有用的帮助信息。
-

理解:

- 查询 (query) 和回复 (reply) 消息: 查询消息和回复消息在格式上是相同的, 但它们通过标识符和标志位来区分。
- 问题部分: 查询部分包括要查询的名称和类型字段。名称是域名, 类型是DNS记录的类型 (如A记录、MX记录等) 。
- 响应部分: 当DNS服务器回复时, 它会返回多个字段:
 - 答案部分 (answers) : 这是服务器对查询的答复, 包含响应的资源记录。
 - 权威部分 (authority) : 提供查询的权威服务器信息。
 - 附加信息 (additional info) : 有时会返回其他有用的信息, 可能对查询有帮助。

这种结构使得DNS可以有效地处理查询和响应, 并确保网络中信息的传递正确高效。